

I2OS Mini Gate:

A Formal Model of Runtime Transition Governance

with Diagrams and Tables

From v0.1 to the v4.1 Design Stage

Masayuki Ando / ANDOM
Independent Researcher, Japan

June 5, 2026

Abstract

Modern AI safety frameworks often focus on output filtering, alignment, reward optimization, or post-hoc evaluation. However, as AI systems increasingly interact with tools, files, APIs, memory, and external environments, safety must be extended from output-level control to action-level governance.

This paper presents a formal model of *I2OS Mini Gate*, a compact runtime governance system developed from version v0.1 to the v4.1 design stage. I2OS Mini Gate treats proposed AI or software actions as state transitions before execution. It classifies them as GO, HOLD, REPAIR, or BLOCK, converts permitted transitions into bounded execution contracts, enforces those contracts during attempted execution, and responds to contract violations through severity classification, containment, repair, rollback, escalation, blocking, and human-verifiable reporting.

The core principle is:

Capability $\neq$ Permission.

The resulting model frames AI safety as pre-execution transition governance, bounded permission during execution, and structured response after violation.

Keywords: AI Safety, Runtime Governance, Transition Governance, Admissibility, Execution Contract, Agentic AI, I2OS, Human-Verifiable Governance

Contents

1	Introduction	3
2	Evolution of I2OS Mini Gate	3
3	State Transition Model	4
4	Admissibility and the Permit Equation	5
5	Admissibility Constraint Decomposition	5
6	Runtime Classification Function	6
7	v3.8 Runtime Governance Refinement	6
8	v4.0: Bounded Permission	7
9	Execution Contract Enforcement	8
10	v4.0 Closed-Loop Governance	8
11	v4.1 Contract Violation Response	9
	11.1 Violation Severity	10
	11.2 Response Selection	11
12	Post-Violation Report	11
13	Integrated Output Function	12
14	General Abstract Model	12
15	Admissible Action Space	13
16	Final Formal Definition	13
17	Ultra-Compact Form	13
18	Conclusion	14

1 Introduction

AI systems are moving from passive text generation toward action-oriented execution. Modern systems may read and write files, call external APIs, send messages, execute commands, modify memory, and delegate tasks to other agents.

This shift creates a new safety problem. It is no longer sufficient to ask whether an AI output is acceptable after generation. Instead, systems must ask whether a proposed action should become real before execution.

Principle 1 (Core Principle).

Capability $\neq$ Permission.

A system should not execute an action simply because it can.

The central claim of this paper is that AI safety should be understood not only as output filtering, but as runtime transition governance.

2 Evolution of I2OS Mini Gate

I2OS Mini Gate evolved from a minimal classifier into a closed-loop runtime governance prototype.

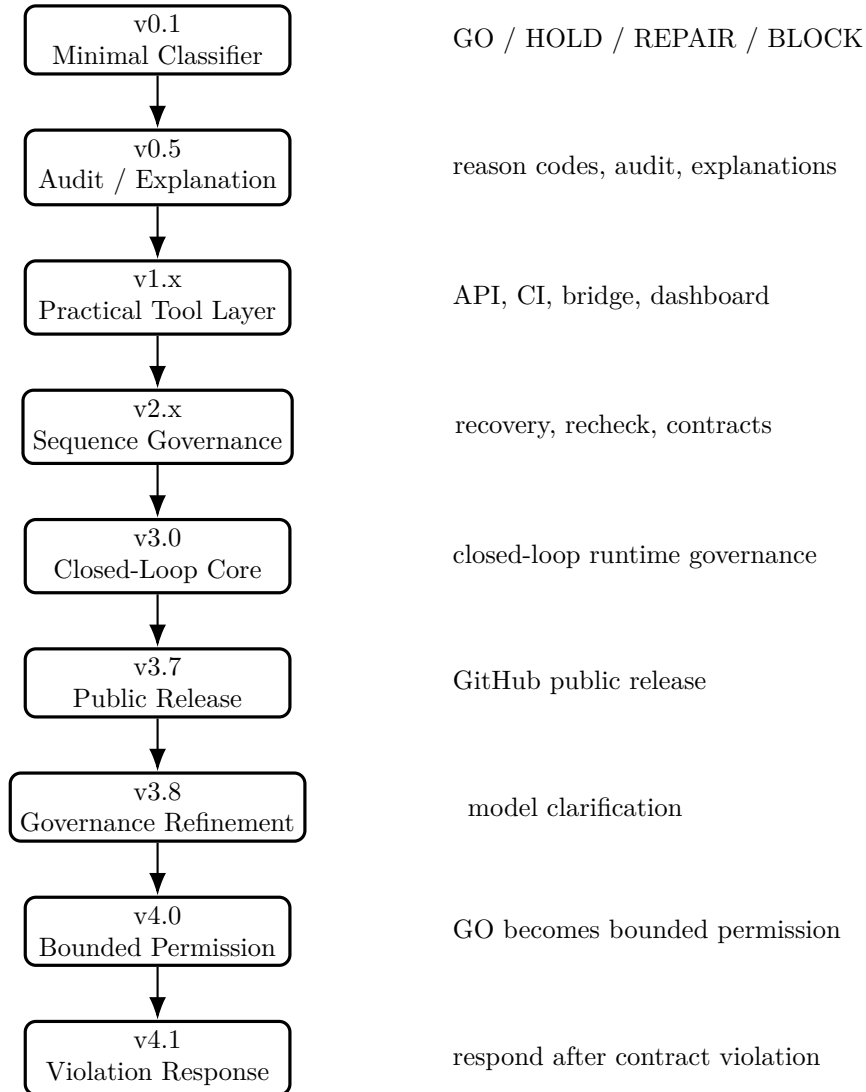


Figure 1: Evolutionary roadmap of I2OS Mini Gate from v0.1 to v4.1.

Version	Role	Structural Meaning
v0.1	Minimal Gate Prototype	Introduced GO/HOLD/REPAIR/BLOCK classification.
v0.5	Audit and Explanation	Added reason codes, risk levels, logs, and human-verifiable explanations.
v1.x	Practical Tool Layer	Added API mode, CI hooks, agent runtime bridge, prompt-injection tests, dashboard, and error handling.
v2.x	Sequence Governance	Extended from single-action checks to recovery paths, recheck loops, execution contracts, and enforcement.
v3.0	Closed-Loop Governance Core	Integrated human-admissibility, recovery, recheck, contract generation, enforcement, and final reporting.
v3.7	Public Release Stabilization	Prepared the repository for public GitHub release.
v3.8	Runtime Governance Refinement	Clarified the transition governance model and example cases.
v4.0	Stable Closed-Loop Runtime Governance	Introduced bounded permission through execution contracts.
v4.1	Contract Violation Response	Defined structured responses when execution violates a contract.

The structural trajectory is:

Classifier → Explainer → Tool → Sequence Governor → Closed-Loop Runtime Governance → Bounded Permission

3 State Transition Model

Let S_t denote the current state of the system. Let T denote a proposed AI or software action. Let S_{t+1} denote the predicted next state after the transition.

A proposed action is modeled as a state transition:

$$T : S_t \rightarrow S_{t+1}.$$

The fundamental question is:

Should T be permitted before execution?

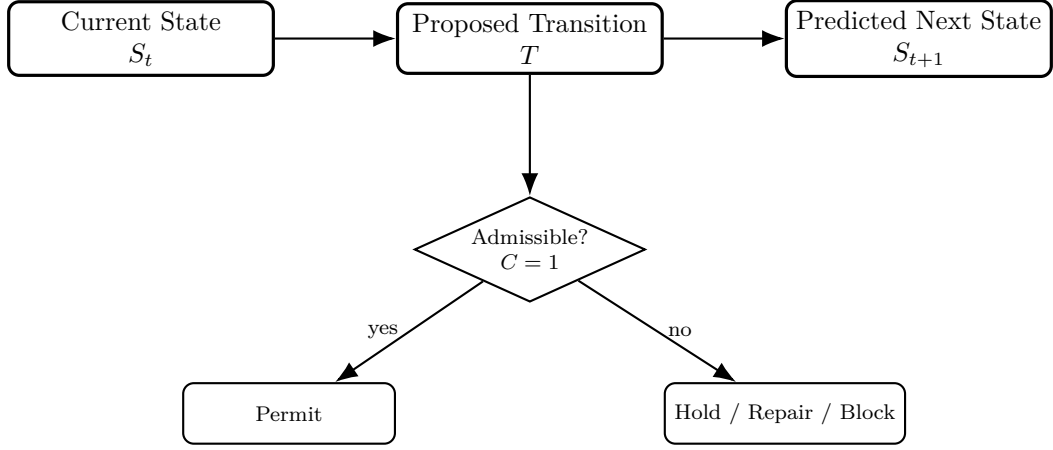


Figure 2: A proposed action is modeled as a state transition before execution.

4 Admissibility and the Permit Equation

Definition 1 (Admissibility Constraint). *An admissibility constraint C determines whether a proposed transition preserves recoverable, synchronized, context-valid, future-compatible continuity.*

The core permit equation is:

$$Permit(T) = \mathbf{1} [C(S_t, T, S_{t+1}) = 1] .$$

Thus:

$$Permit(T) = 1$$

if and only if the transition from S_t to S_{t+1} through T satisfies the admissibility constraint.

5 Admissibility Constraint Decomposition

The admissibility constraint is multi-layered:

$$C(S_t, T, S_{t+1}) = C_{\text{context}} \wedge C_{\text{safety}} \wedge C_{\text{recovery}} \wedge C_{\text{sync}} \wedge C_{\text{future}} \wedge C_{\text{human}} \wedge C_{\text{scope}} \wedge C_{\text{audit}} .$$

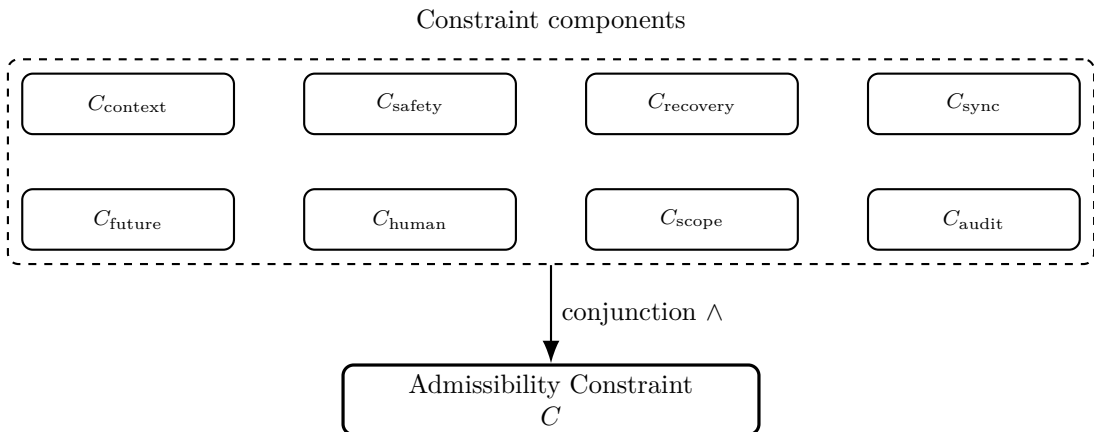


Figure 3: The admissibility constraint C is a conjunction of multiple governance constraints.

Constraint	Meaning
C_{context}	The transition is valid in the current context.
C_{safety}	The transition satisfies safety requirements.
C_{recovery}	The transition is recoverable if failure occurs.
C_{sync}	The transition is synchronized with user intent and task scope.
C_{future}	The transition does not increase future instability or collapse risk.
C_{human}	Required human confirmation is present.
C_{scope}	The transition remains within the permitted scope.
C_{audit}	The transition can be logged, explained, and audited.

6 Runtime Classification Function

Let the runtime classification function be:

$$G(T) \in \{\text{GO}, \text{HOLD}, \text{REPAIR}, \text{BLOCK}\}.$$

Table 3: Runtime classification outputs.

Output	Meaning
GO	The transition is admissible. In v4.0 and later, it must also be bounded by an execution contract.
HOLD	The transition requires more information, clarification, authorization, or confirmation.
REPAIR	The transition is currently inadmissible but may become admissible through a repair mapping.
BLOCK	The transition is inadmissible, unsafe, unrecoverable, or not repairable.

$$G(T) = \text{GO} \iff C(S_t, T, S_{t+1}) = 1.$$

$$G(T) = \text{HOLD} \iff C_{\text{core}}(T) = 1 \wedge C_{\text{human}}(T) = 0.$$

Let $\rho : T \mapsto T'$ denote a repair mapping.

$$G(T) = \text{REPAIR} \iff C(T) = 0 \wedge \exists \rho : C(\rho(T)) = 1.$$

$$G(T) = \text{BLOCK} \iff C(T) = 0 \wedge \neg \exists \rho : C(\rho(T)) = 1.$$

7 v3.8 Runtime Governance Refinement

v3.8 clarifies the governance pipeline:

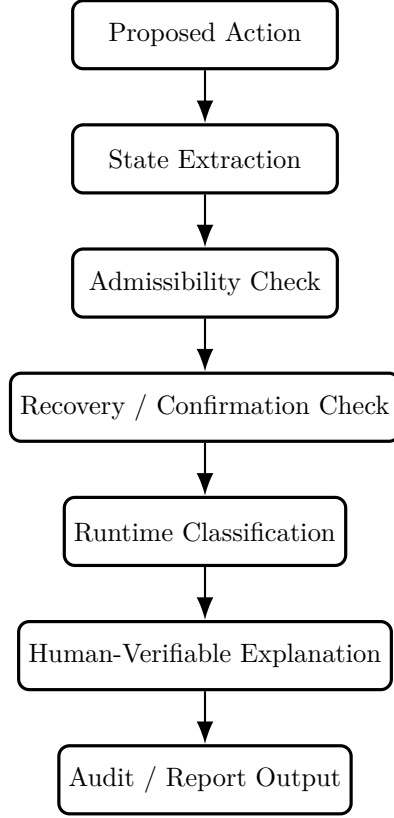


Figure 4: v3.8 runtime governance refinement pipeline.

This can be modeled as function composition:

$$G(T) = \textit{Classify} \circ \textit{Confirm} \circ \textit{Recover} \circ \textit{Check} \circ \textit{Extract}(S_t, T).$$

Define:

$$X_t = \textit{Extract}(S_t, T), \quad A_t = \textit{Check}_C(X_t), \quad R_t = \textit{Recover}(A_t, T),$$

$$H_t = \textit{Confirm}(R_t), \quad G_t = \textit{Classify}(H_t),$$

$$\textit{Report}_t = \textit{Explain}(G_t, X_t, C, R_t, H_t).$$

8 v4.0: Bounded Permission

v4.0 introduces the principle:

GO $\neq$ Unlimited Permission.

Instead:

GO = Bounded Permission.

Let K denote an execution contract.

$$K = \{\textit{allowed_action}, \textit{allowed_target}, \textit{allowed_scope}, \textit{prohibited_actions}, \textit{preconditions}, \textit{audit_required}, \textit{hu}$$

$$\text{GO}(T) \Rightarrow \exists K : \text{Bound}(T, K) = 1.$$

Therefore:

$$\text{BoundedPermit}(T, K) = \mathbf{1} [C(S_t, T, S_{t+1}) = 1 \wedge \text{Bound}(T, K) = 1].$$

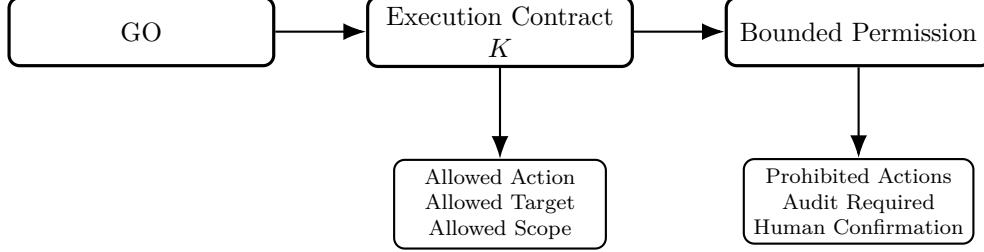


Figure 5: In v4.0, GO is converted into bounded permission through an execution contract.

9 Execution Contract Enforcement

Let E denote the attempted execution. Let K denote the issued execution contract.

$$\text{Enforce}(E, K) \in \{\text{PASS}, \text{VIOLATION}\}.$$

$$\text{Enforce}(E, K) = \text{PASS} \iff E \subseteq K.$$

$$\text{Enforce}(E, K) = \text{VIOLATION} \iff E \not\subseteq K.$$

Table 4: Contract enforcement examples.

Contract K	Attempt E	Result
Read $file_A$	Read $file_A$	PASS
Read $file_A$	Read $file_A$ + Upload $file_A$	VIOLATION
Write $summary.md$	Overwrite $README.md$	VIOLATION
Delete confirmed temp files	Delete project directory	VIOLATION

10 v4.0 Closed-Loop Governance

The v4.0 closed-loop governance flow is:

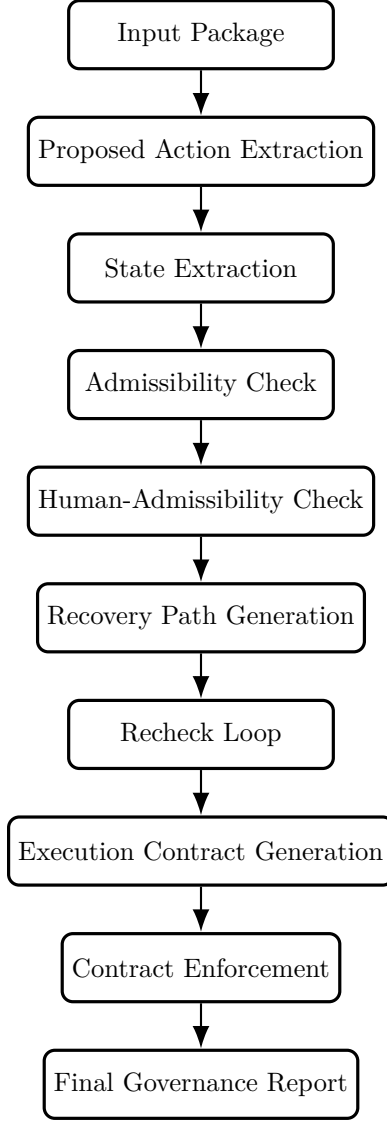


Figure 6: v4.0 stable closed-loop runtime governance pipeline.

Define:

$$X = \text{Extract}(S_t, T), \quad a = C(X), \quad r = \text{Recovery}(T),$$

$$T' = \text{Recheck}(r(T)), \quad K = \text{Contract}(T'), \quad e = \text{Enforce}(E, K),$$

$$\text{FGR} = \text{FinalGovernanceReport}(X, a, r, T', K, e).$$

11 v4.1 Contract Violation Response

v4.1 defines how the system responds when execution violates the issued contract.

Let:

$$V = \text{Violation}(E, K).$$

Let severity be:

$$\sigma(V) \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}, \text{CRITICAL}\}.$$

Let response selection be:

$$\lambda(V) \in \{\text{OBSERVE}, \text{PAUSE}, \text{REPAIR}, \text{ROLLBACK}, \text{ESCALATE}, \text{BLOCK}\}.$$

If:

$$\text{Enforce}(E, K) = \text{VIOLATION},$$

then:

$$\lambda(V) = \text{SelectResponse}(\sigma(V), \text{Context}, \text{Policy}).$$

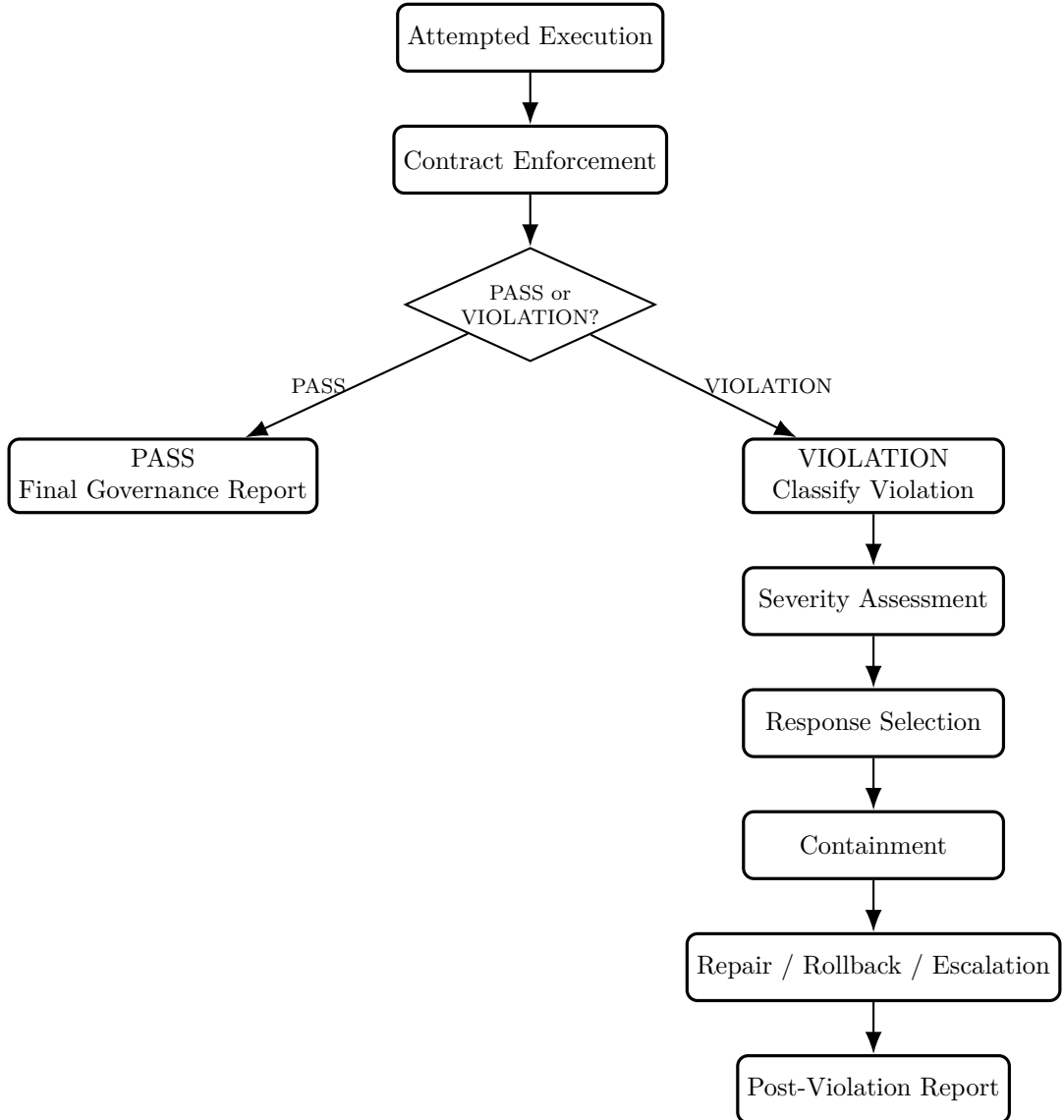


Figure 7: v4.1 contract violation response flow.

11.1 Violation Severity

Violation risk can be modeled as:

$$Risk(V) = w_d D + w_e E_{\text{ext}} + w_c Cred + w_g Gov + w_i Irrev + w_s Scope + w_h Human.$$

Severity is thresholded as:

$$\sigma(V) = \begin{cases} \text{LOW}, & Risk(V) < \theta_1, \\ \text{MEDIUM}, & \theta_1 \leq Risk(V) < \theta_2, \\ \text{HIGH}, & \theta_2 \leq Risk(V) < \theta_3, \\ \text{CRITICAL}, & Risk(V) \geq \theta_3. \end{cases}$$

11.2 Response Selection

$$\lambda(V) = SelectResponse(\sigma(V), Recoverable(V), ExternalEffect(V), CredentialRisk(V), GovernanceBypass(V))$$

Table 5: Severity-to-response matrix.

Severity	Typical Condition	Default Response
LOW	Minor boundary deviation without state change	OBSERVE/PAUSE
MEDIUM	Scope expansion or unsafe modification attempt	PAUSE/REPAIR
HIGH	External effect, destructive attempt, sensitive action	ESCALATE/BLOCK
CRITICAL	Credential exposure, destructive deletion, governance bypass	BLOCK

12 Post-Violation Report

A post-violation report is generated when contract enforcement fails.

$$PVR = ReportViolation(K, E, V, \sigma(V), \lambda(V), Containment, Rollback, Repair, Audit).$$

Its content is represented as:

$$PVR = \{contract_id, attempted_action, allowed_action, violated_rule, severity, response_level, containment\}$$

Table 6: Post-violation report fields.

Field	Meaning
contract_id	Identifier of the issued execution contract.
attempted_action	Action attempted during execution.
allowed_action	Action originally allowed by the contract.
violated_rule	Contract rule that was violated.
severity	LOW / MEDIUM / HIGH / CRITICAL.
response_level	OBSERVE / PAUSE / REPAIR / ROLLBACK / ESCALATE / BLOCK.
containment_status	How execution was contained.
rollback_status	Whether rollback was possible or required.
repair_option	Possible safer path, if available.
audit_trace	Human-verifiable record of governance events.

13 Integrated Output Function

Let FGR denote the Final Governance Report. Let PVR denote the Post-Violation Report.

$$Output(T) = \begin{cases} FGR(T, K, E), & Enforce(E, K) = PASS, \\ PVR(T, K, E, V), & Enforce(E, K) = VIOLATION. \end{cases}$$

14 General Abstract Model

I2OS Mini Gate can be abstractly defined as:

$$MG = \langle S, T, C, G, \rho, K, E, V, \sigma, \lambda, R \rangle.$$

Symbol	Meaning
S	State space.
T	Set of possible transitions.
C	Admissibility constraint.
G	Classification function: GO/HOLD/REPAIR/BLOCK.
ρ	Repair mapping.
K	Execution contract.
E	Attempted execution.
V	Contract violation.
σ	Violation severity function.
λ	Violation response function.
R	Report generation function.

The full operation can be expressed as:

$$I2OS_MG(T) = R(\lambda(\sigma(V(E, K(\rho(C(Extract(S_t, T))))))))).$$

Extract $\rightarrow$ *Check* $\rightarrow$ *Classify* $\rightarrow$ *Repair* $\rightarrow$ *Contract* $\rightarrow$ *Enforce* $\rightarrow$ *Respond* $\rightarrow$ *Report*.

15 Admissible Action Space

Let $\mathcal{T}$ be the set of possible transitions.

The admissible transition set is:

$$\mathcal{T}_{allow} = \{T \in \mathcal{T} \mid C(S_t, T, S_{t+1}) = 1\}.$$

The executable transition set under v4.0 and v4.1 is:

$$\mathcal{T}_{execute} = \{T \in \mathcal{T} \mid C(T) = 1 \wedge Bound(T, K) = 1 \wedge Enforce(E, K) = PASS\}.$$

If:

$$E \not\subseteq K,$$

then:

$$E \not\subseteq K \Rightarrow V \Rightarrow \sigma(V) \Rightarrow \lambda(V) \Rightarrow PVR.$$

16 Final Formal Definition

Definition 2 (I2OS Mini Gate). *I2OS Mini Gate is a closed-loop runtime governance system that extracts proposed AI/software actions as state transitions before execution, classifies them according to admissibility constraints, converts permitted transitions into bounded execution contracts, enforces those contracts during attempted execution, and responds to contract violations through severity classification, containment, repair, rollback, escalation, blocking, and human-verifiable reporting.*

Formally:

$$I2OS_MG : (S_t, T) \mapsto \begin{cases} \text{FGR}, & C(T) = 1 \wedge Bound(T, K) = 1 \wedge E \subseteq K, \\ \text{PVR}, & E \not\subseteq K, \\ \text{HOLD}, & C_{core}(T) = 1 \wedge C_{human}(T) = 0, \\ \text{REPAIR}, & C(T) = 0 \wedge \exists \rho : C(\rho(T)) = 1, \\ \text{BLOCK}, & C(T) = 0 \wedge \neg \exists \rho : C(\rho(T)) = 1. \end{cases}$$

17 Ultra-Compact Form

The shortest expression of I2OS Mini Gate is:

$$\text{Capability} \neq \text{Permission}.$$

Permission is defined as:

$$\text{Permission} = \text{Admissibility} \wedge \text{Boundedness} \wedge \text{Recoverability} \wedge \text{Human Verifiability}.$$

A safe AI action is:

$$\text{Safe AI Action} = \text{Permitted Transition} + \text{Bounded Contract} + \text{Violation Response}.$$

Equivalently:

$$\text{Safe AI Action}(T) = C(T) \wedge Bound(T, K) \wedge Response(V).$$

18 Conclusion

I2OS Mini Gate formalizes a shift from output filtering to runtime transition governance.

The system does not merely ask whether an AI can act. It asks whether a proposed transition is admissible before execution, whether permission is bounded during execution, and how governance should respond after violation.

Thus:

AI Safety = Pre-Execution Governance + Bounded Execution + Post-Violation Response.

In natural language:

AI safety is not only output filtering.

It is admissible transition governance before action, bounded permission during execution, and structured response after violation.

Safe intelligence is not intelligence that never collapses.

Safe intelligence is intelligence that can recursively re-synchronize.